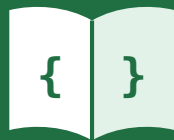

Capstone — Bookstore Platform

Architecture and Decisions

Spring Boot Edu



2026-09-25

Contents

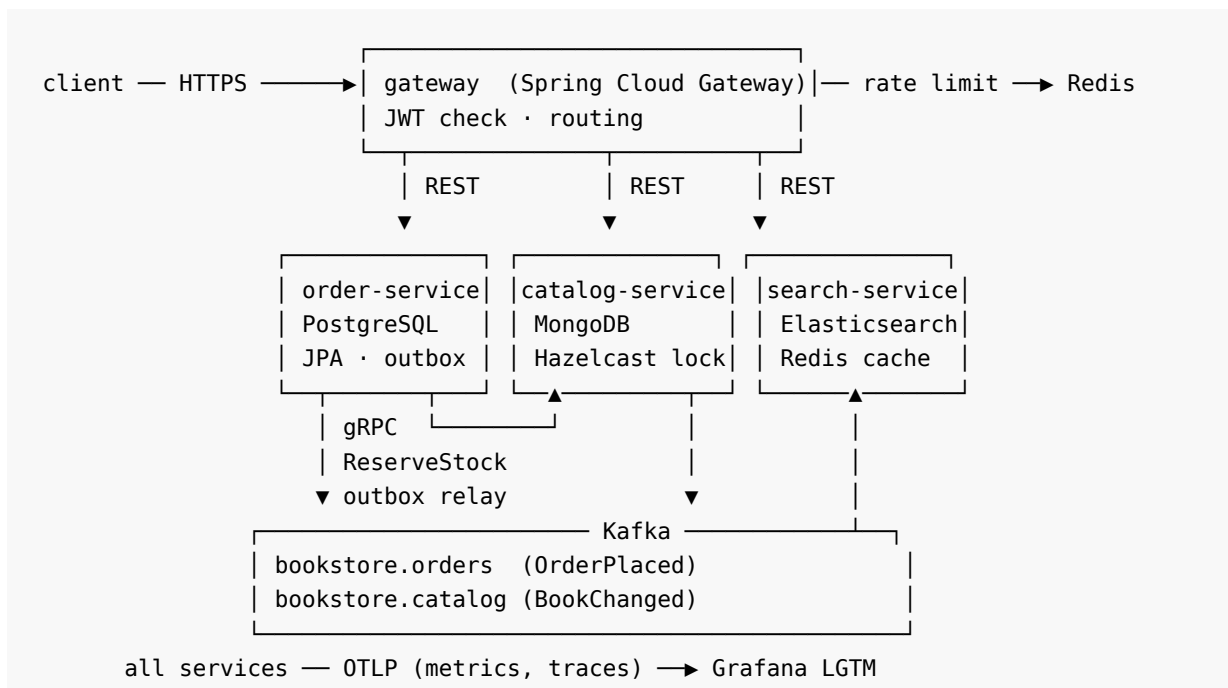
1. Goal	2
2. Overview	2
3. The Main Flows	3
3.1 Placing an Order	3
3.2 Changing the Catalog	4
3.3 Searching	4
4. Architecture Decision Records	4
ADR-1: Four Services and a Gateway	4
ADR-2: Stock Reservation Synchronously over gRPC	4
ADR-3: Transactional Outbox for OrderPlaced	5
ADR-4: One Database per Service, Search as a Read Model	5
ADR-5: JWT at the Gateway and in the Services	5
ADR-6: Hazelcast IMap Lock for Stock	5
ADR-7: Redis for Cache and Rate Limiting	6
ADR-8: Deployment with Docker Compose and Helm	6
5. Observability	6
6. Project Layout	6

1. Goal

The capstone combines the technologies of the course in one working system: an online bookstore where customers search books, place orders, and the system keeps stock, search index and notifications consistent. It runs with one command on Docker Compose, and on a local Kubernetes cluster (kind) with Helm.

Success means (SPEC, success criterion 6): `docker compose up --build` starts everything; one order touches PostgreSQL, MongoDB, Elasticsearch, Redis, Kafka and Hazelcast; an end-to-end test proves the flow `order → event → search index → cache`.

2. Overview



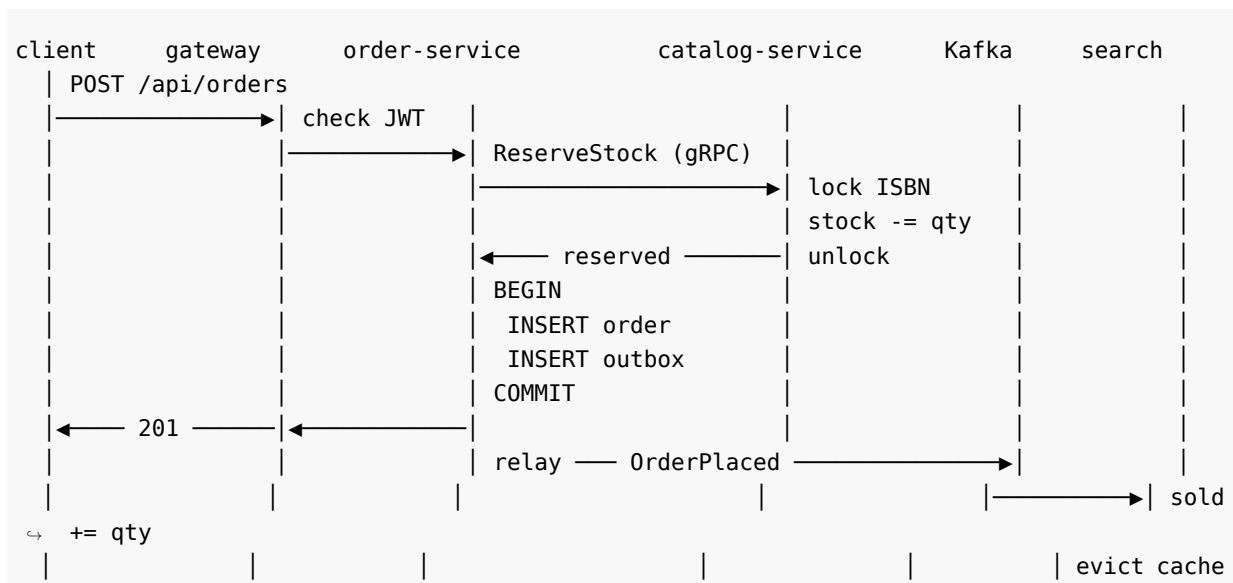
Service	Owns	Technologies	Course modules
gateway	the entry point	Spring Cloud Gateway, JWT resource server, Redis rate limiter	12, 23

Service	Owns	Technologies	Course modules
order-service	orders	Spring Data JPA on PostgreSQL, Flyway, transactional outbox, gRPC client	06, 11, 21
catalog-service	books and stock	Spring Data MongoDB, gRPC server, Hazelcast IMap lock, Kafka producer	07, 09, 11, 21
search-service	the search read model	Spring Data Elasticsearch, Redis cache, Kafka consumer	08, 10, 11
contracts (library)	the shared contracts	.proto files, event records	21

Every service has its own database. No service reads another service's database.

3. The Main Flows

3.1 Placing an Order



1. The gateway checks the JWT and forwards the request.
2. The order service reserves stock **synchronously** over gRPC: the customer must know at once whether the book is available.
3. The catalog service locks the ISBN in Hazelcast, so that two orders cannot sell the last copy twice (also with several catalog instances).
4. The order and an outbox row are written in **one** transaction. A relay publishes the outbox rows to Kafka.
5. Consumers react independently: the search service updates the book's sales numbers and evicts its cache; the order service's notification listener records the confirmation.

3.2 Changing the Catalog

The catalog service publishes `BookChanged` to `bookstore.catalog` whenever a book is created or changed (also for its seed data at startup). The search service consumes these events and (re)indexed the book. The search index is a **read model**: it can always be rebuilt from the events.

3.3 Searching

`GET /api/search?q=...` goes through the gateway to the search service. Results are cached in Redis for a short time. Index updates evict the affected entries.

4. Architecture Decision Records

ADR-1: Four Services and a Gateway

- **Context:** the course must show distributed communication (REST, gRPC, Kafka) and several databases, but a student must still be able to run everything on a laptop.
- **Decision:** gateway + order-service, catalog-service, search-service, plus a shared contracts library (SPEC decision 13).
- **Consequences:** every technology has a clear owner. Four JVMs plus infrastructure need about 8 GB of Docker memory. A modular monolith (module 18) would be simpler to operate; the capstone chooses services on purpose, to practise their problems.

ADR-2: Stock Reservation Synchronously over gRPC

- **Context:** an order may only be accepted when the book is in stock.
- **Decision:** order-service calls `ReserveStock` on catalog-service over gRPC, with a deadline of 2 seconds.

- **Alternatives:** an asynchronous saga (reserve by event, confirm or cancel later) scales better and survives catalog outages, but makes the customer wait for a second answer.
- **Consequences:** the order service depends on the catalog at order time. The deadline turns a catalog outage into a clear 503 instead of hanging requests; a short restart is bridged by retrying the idempotent reservation (exercise 3). A circuit breaker (module 23) would additionally stop calling a catalog that stays down.

ADR-3: Transactional Outbox for OrderPlaced

- **Context:** writing the order and publishing the event must not get out of sync (module 11).
- **Decision:** the event is written to an outbox table in the order's transaction; a scheduled relay publishes and marks it.
- **Consequences:** at-least-once delivery; consumers must be idempotent (the search service keys updates by order ID).

ADR-4: One Database per Service, Search as a Read Model

- **Decision:** PostgreSQL for orders (transactions, constraints), MongoDB for the catalog (flexible book documents), Elasticsearch only as a derived read model fed by events.
- **Consequences:** no joins across services; data other services need travels in events or gRPC responses.

ADR-5: JWT at the Gateway and in the Services

- **Decision:** the gateway validates the bearer token and forwards it; every service is also a resource server (defence in depth). For the local system, the gateway offers a development token endpoint with demo users, and tokens are signed with an HMAC key from the environment.
- **Production:** an authorization server (Spring Authorization Server, module 12, or Keycloak) with asymmetric keys (JWKS).

ADR-6: Hazelcast IMap Lock for Stock

- **Decision:** `IMap.tryLock(isbn)` around the stock update (SPEC decision 10: the CP subsystem is Enterprise-only).
- **Consequences:** correct with several catalog instances; the lock has a lease time, so a crashed instance cannot block an ISBN forever.

ADR-7: Redis for Cache and Rate Limiting

- **Decision:** the search service caches search results in Redis; the gateway's rate limiter keeps its token buckets in Redis.
- **Consequences:** one more infrastructure service, but both uses need shared state across instances.

ADR-8: Deployment with Docker Compose and Helm

- **Decision:** one capstone/compose.yaml for the whole system, and a Helm chart for kind. Images use the multi-stage jlink Dockerfile of module 20.
- **Consequences:** the same images run in both environments; configuration comes from environment variables (12-factor).

5. Observability

All services export metrics and traces over OTLP to Grafana LGTM (module 15). The trace context travels over HTTP, gRPC and Kafka, so one order is one trace from the gateway to the search index.

6. Project Layout

capstone/	
pom.xml	aggregator
contracts/	.proto files and event records (a library)
gateway/	Spring Cloud Gateway
order-service/	
catalog-service/	
search-service/	
e2e-tests/	the end-to-end test (Testcontainers)
Dockerfile	one image recipe for the four services (jar from source or from
↪ the host)	
compose.yaml, up.sh	the whole system
k8s/helm/bookstore/	the Helm chart; k8s/deploy-kind.sh installs it on kind
exercise/, solution/	the capstone exercises (a copy of order-service each)
requests.http	HTTP examples through the gateway
docs/	this document, the capstone guide and exercises (TR + EN, PDF)